

東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

The case for zero-copy in containers: Accelerating KubeVirt and enabling memory providers

Anton Protopopov (Isovalent)
Daniel Borkmann (Isovalent)
David Wei (Meta)

Linux Plumbers 2025

ISOVALENT
now part of cisco

Meta

COMPUTING

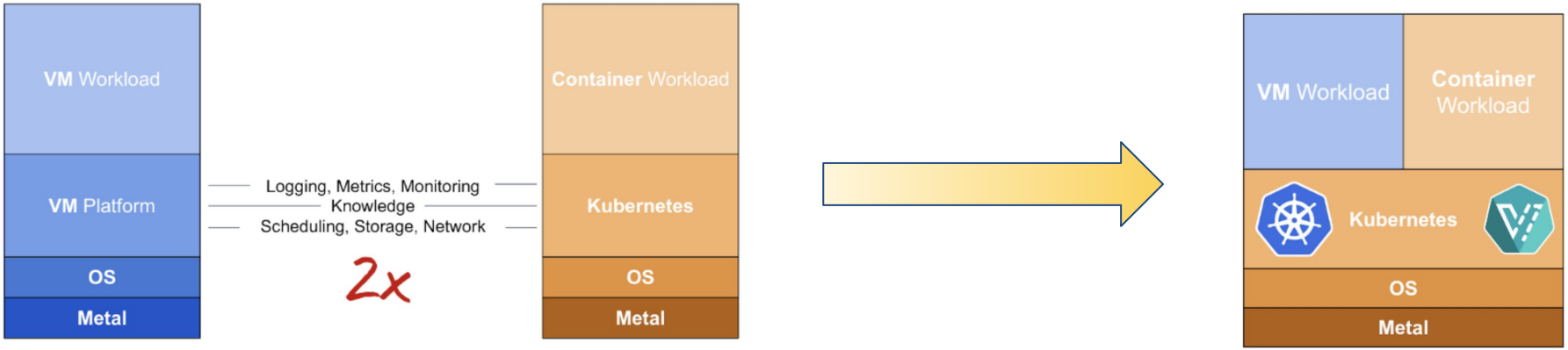
Turning migration into modernization

The VMware shake-up has led to an IT inflection point. Leaders are now weighing whether to renew, migrate, or redesign entirely for the cloud era.

By MIT Technology Review Insights

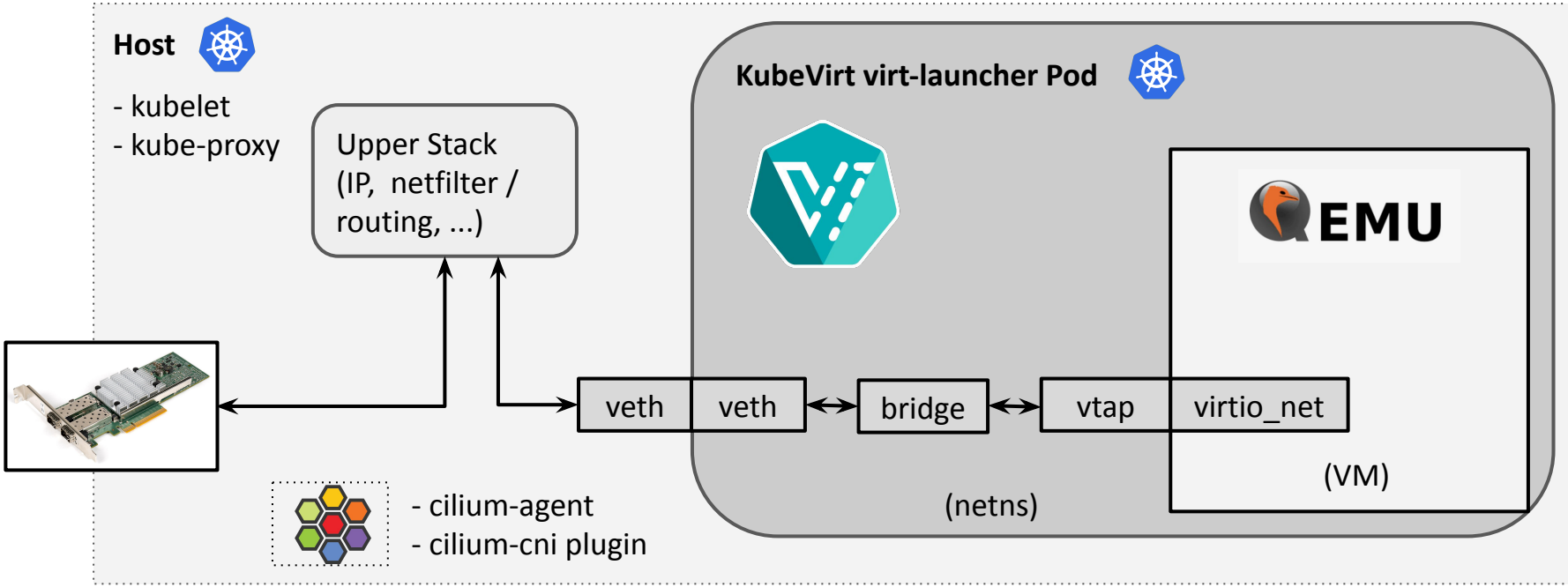
October 2, 2025

Kubernetes & KubeVirt enables cost savings through infrastructure convergence:



[img: kubevirt.io]

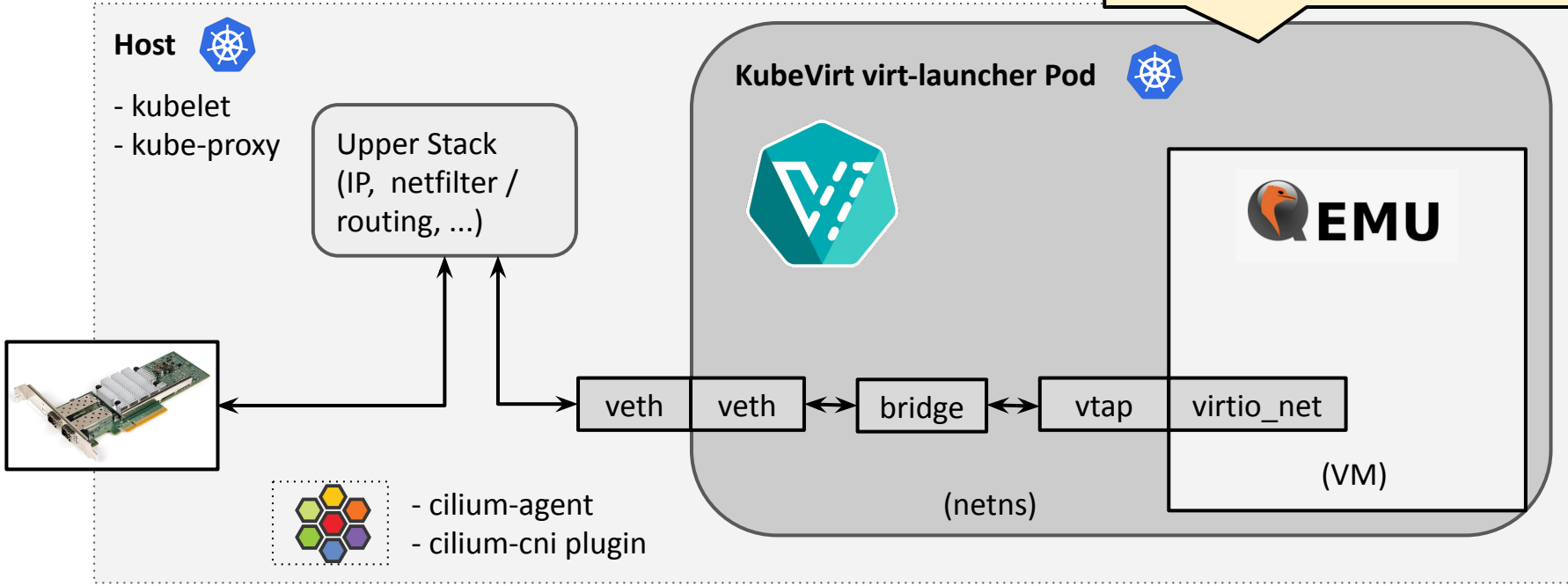
Standard KubeVirt Pod Datapath:



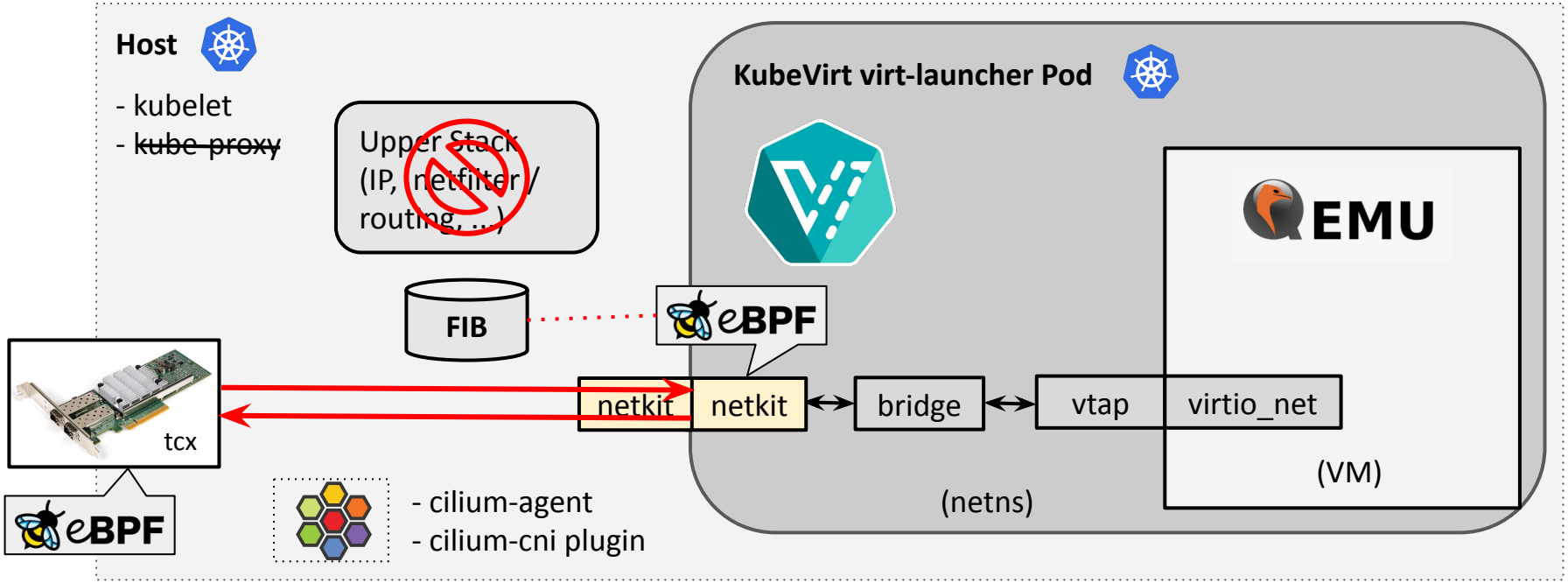
Standard KubeVirt Pod Datapath:

Fyi: virt-launcher Pod provides ...
- cgroups
- namespaces

... for hosting the VM process



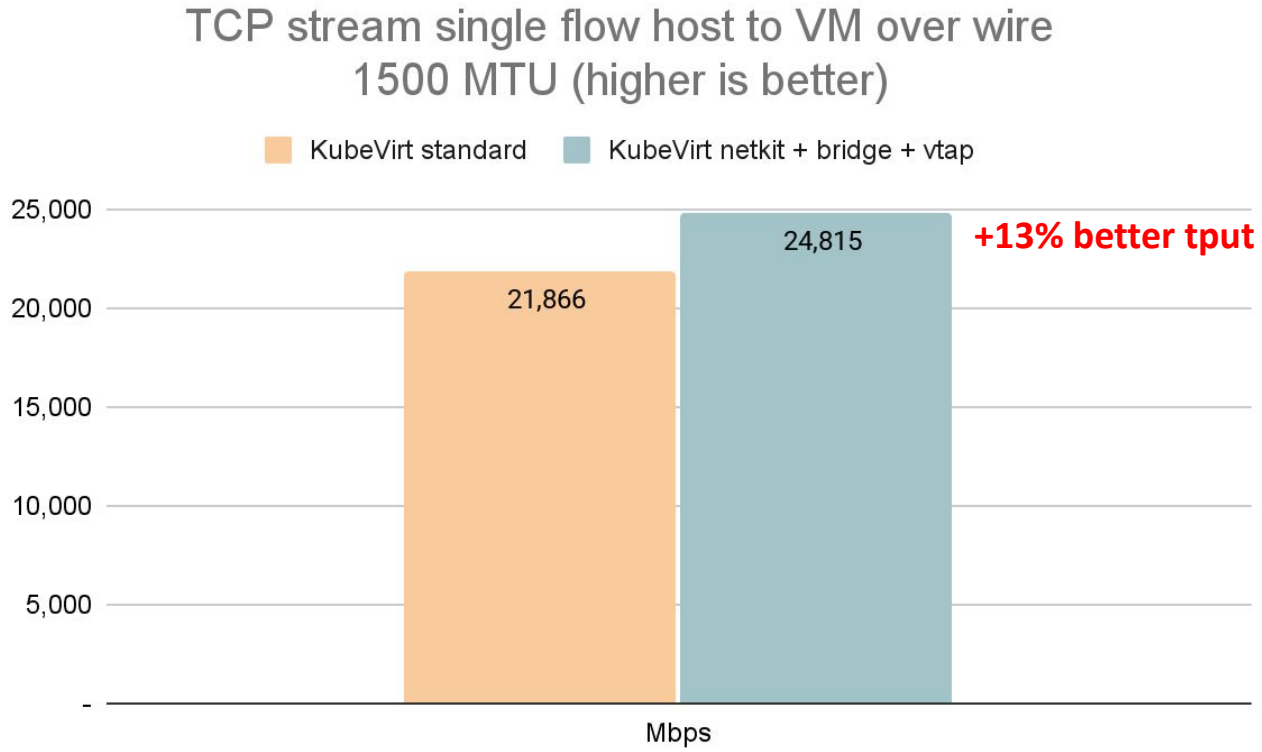
Cilium's KubeVirt Pod Datapath:



Cilium's KubeVirt Pod Datapath:

Building Blocks:
- BPF kube-proxy replacement
- BPF Host Routing
- BIG TCP for GRO/GSO/TSO
- tcx-based BPF datapath layer
- netkit devices for Pods

(... but still KubeVirt with bridge + vtap)



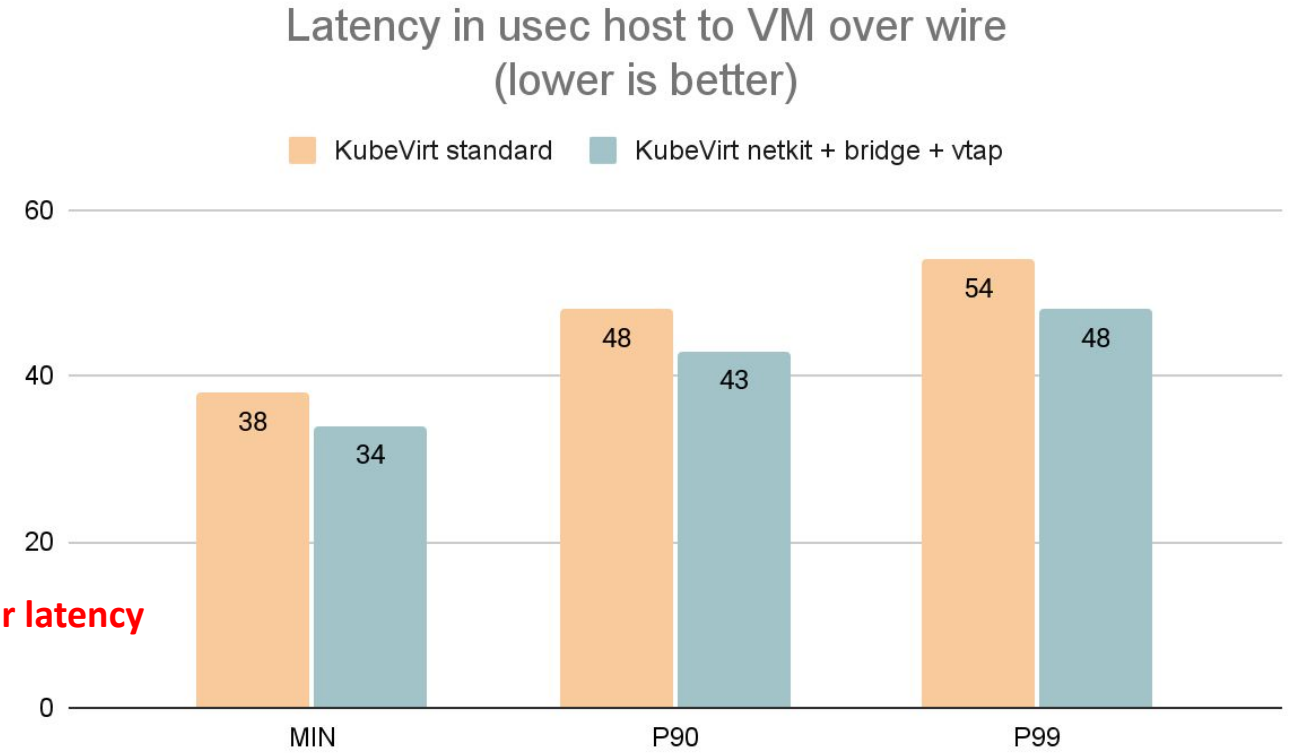
Back to back: AMD Ryzen 9 3950X @ 3.5 GHz, 128G RAM @ 3.2 GHz, PCIe 4.0, ConnectX-6 Dx, mlx5 driver, striding mode, LRO off, 1500 MTU
netperf -t TCP_RR -H <VM IP> -- -O MIN_LATENCY,P90_LATENCY,P99_LATENCY,THROUGHPUT

Cilium's KubeVirt Pod Datapath:



Building Blocks:
- BPF kube-proxy replacement
- BPF Host Routing
- BIG TCP for GRO/GSO/TSO
- tcx-based BPF datapath layer
- netkit devices for Pods

(... but still KubeVirt with bridge + vtap)

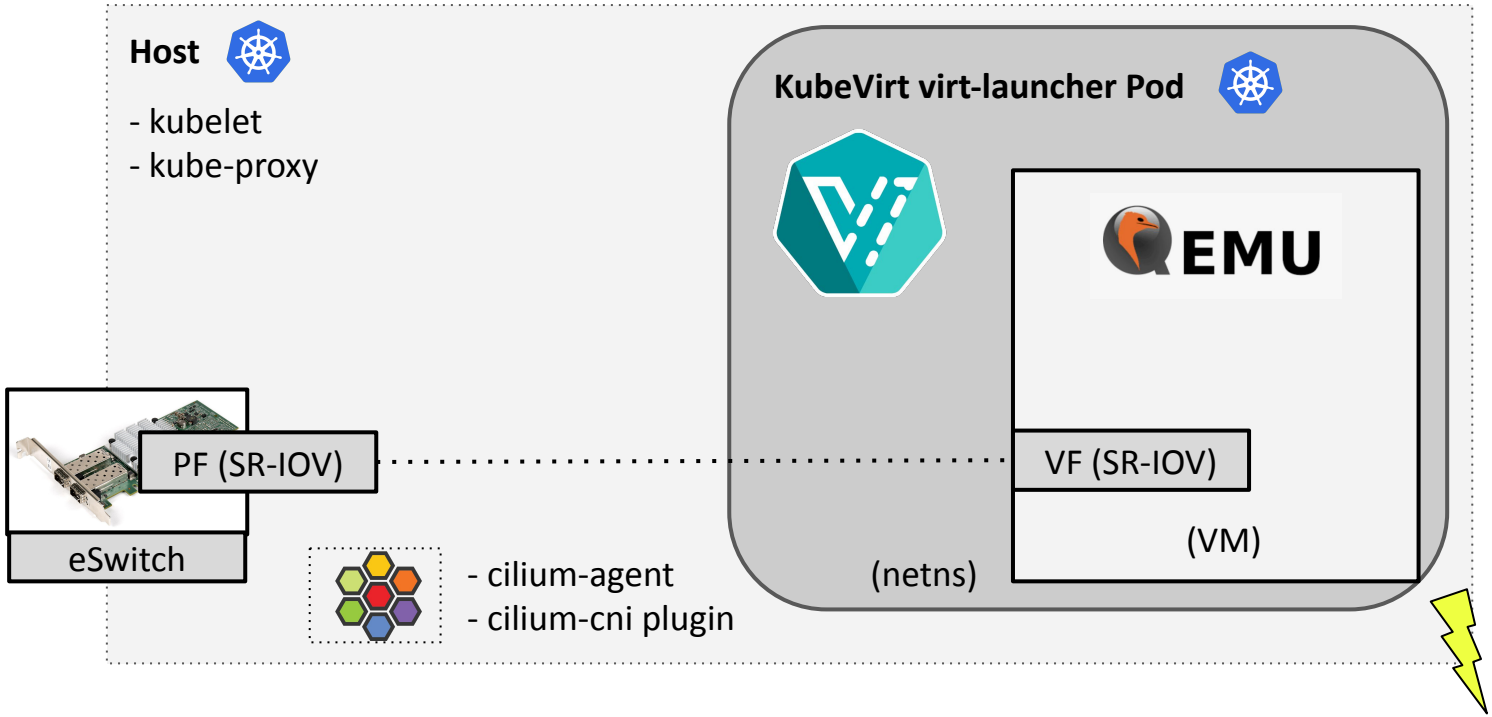


12% lower latency

Back to back: AMD Ryzen 9 3950X @ 3.5 GHz, 128G RAM @ 3.2 GHz, PCIe 4.0, ConnectX-6 Dx, mlx5 driver, striding mode, LRO off, 1500 MTU
netperf -t TCP_RR -H <VM IP> -- -O MIN_LATENCY,P90_LATENCY,P99_LATENCY,THROUGHPUT

Removal of the netns overhead helps a bit, but the main overhead is still on the VM side.

What about SR-IOV?

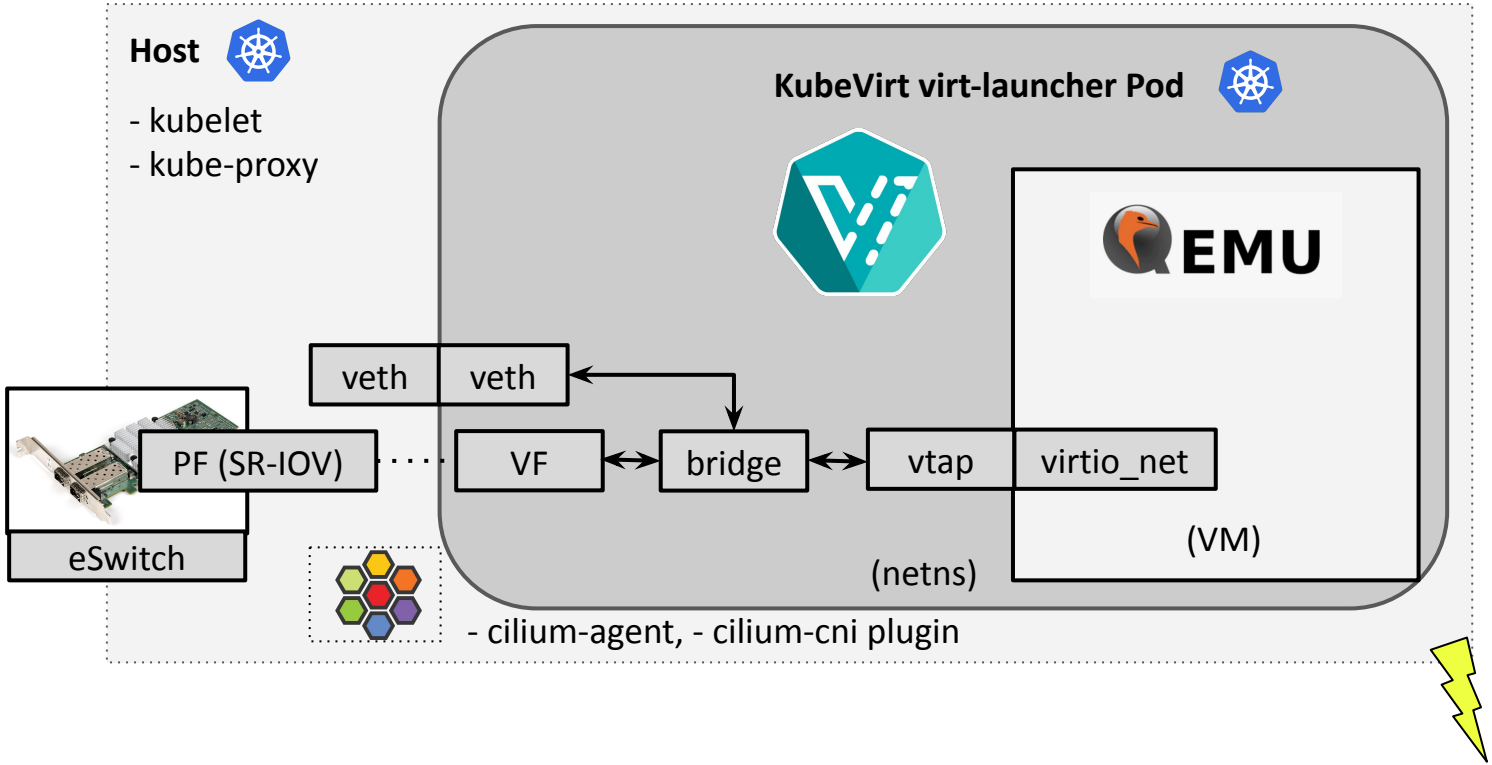


Fast:
- HW virtualization
- VFs are lightweight PCIe functions

Issues:
- Completely hidden from host networking stack
- No observability
- No policy enforcement
- E/W K8s service access?
- Capabilities depend on underlying hardware

No unified container / VM Kubernetes stack.

What about SR-IOV?



Issues:

- PF/VF setup here only solves the netns overhead
- HW-centric/dependency
- This is already solved with the netkit setup
- More complex setup here compared to netkit given we would have to leak routes into virt-launcher Pod instead of reusing the hostns routing table

No real gain over netkit, more complex setup

Other alternatives?

QEMU / QEMU / Commits / **cb039ef3**

Commit `cb039ef3` authored 1 year ago by **Ilya Maximets** Committed by **Jason Wang** 1 year ago

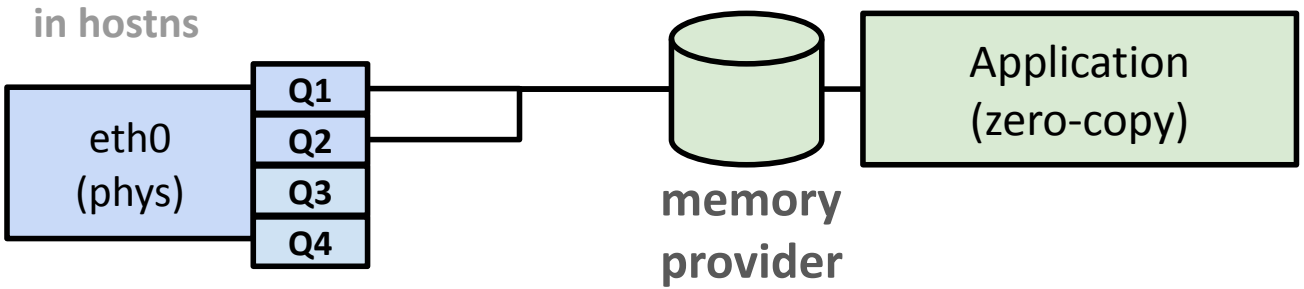
net: add initial support for AF_XDP network backend

```
AF_XDP is a network socket family that allows communication directly
with the network device driver in the kernel, bypassing most or all
of the kernel networking stack.  In the essence, the technology is
pretty similar to netmap.  But, unlike netmap, AF_XDP is Linux-native
and works with any network interfaces without driver modifications.
Unlike vhost-based backends (kernel, user, vdpa), AF_XDP doesn't
require access to character devices or unix sockets.  Only access to
the network interface itself is necessary.
```

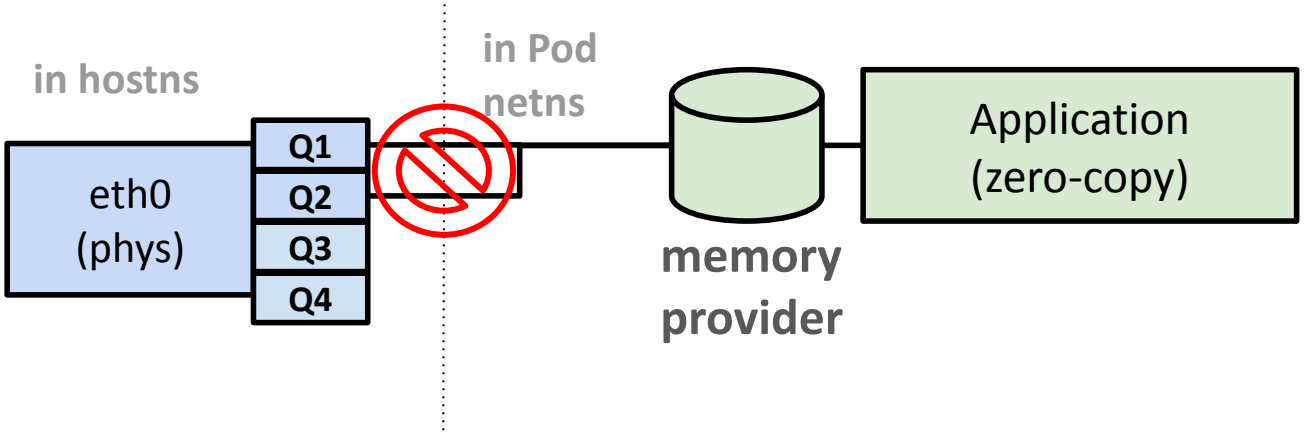
There’s a catch: KubeVirt launches QEMU in its own Pod (netns). No AF_XDP support in that netns given it needs to attach to a phys device.

The same holds true for other zero-copy mechanisms such as io_uring zero-copy and TCP devmem ...

Recap: Memory provider zero copy



Problem: No access to phys NIC from Pods/netns

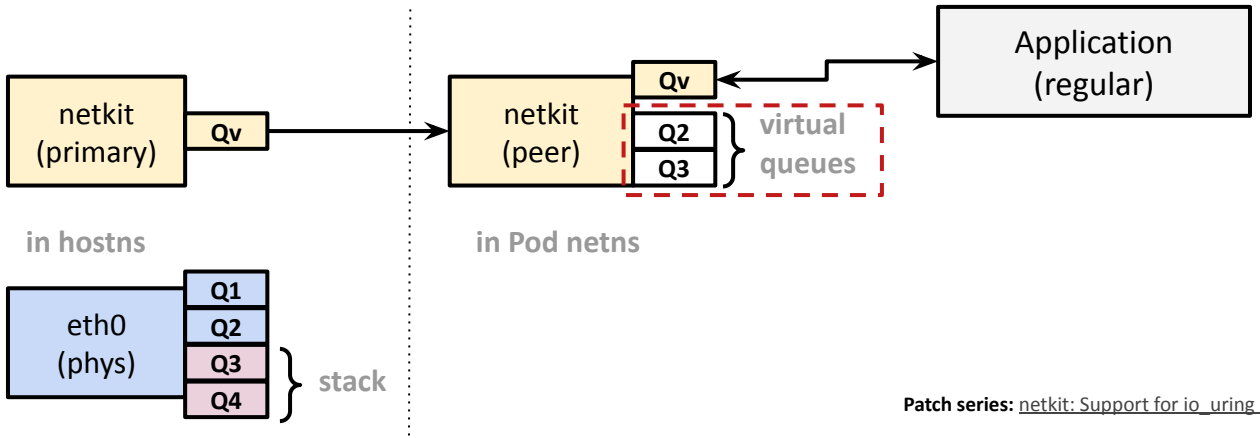


Proposal: netkit & Hardware Queue Leasing for Pods



netkit extension for *native* zero-copy performance inside Pods:

- Create netkit peer with numrxqueues > 1 as *virtual queues*



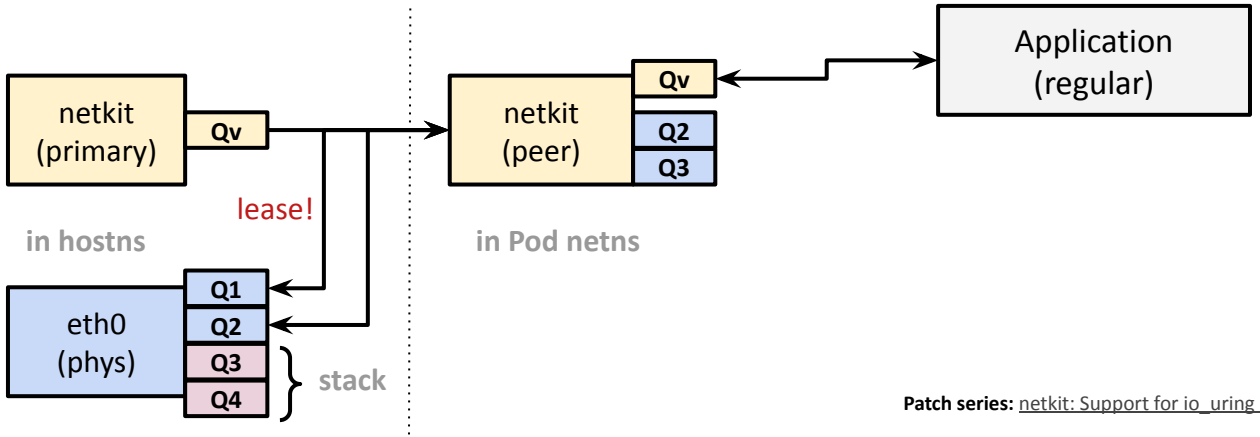
Patch series: netkit: Support for io_uring zero-copy and AF_XDP (Daniel Borkmann, David Wei)

Proposal: netkit & Hardware Queue Leasing for Pods



netkit extension for *native* zero-copy performance inside Pods:

- Create netkit peer with numrxqueues > 1 as *virtual queues*
- Lease physical queues in the physical NIC to virtual queues in netkit



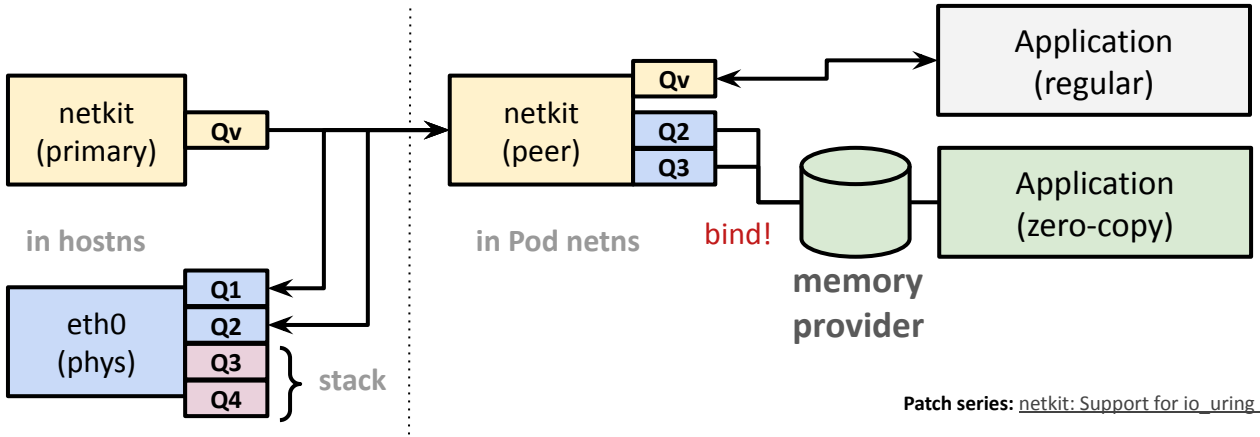
Patch series: netkit: Support for io_uring zero-copy and AF_XDP (Daniel Borkmann, David Wei)

Proposal: netkit & Hardware Queue Leasing for Pods



netkit extension for *native* zero-copy performance inside Pods:

- Create netkit peer with numrxqueues > 1 as *virtual queues*
- Lease physical queues in the physical NIC to virtual queues in netkit
- The netkit device is moved into the target Pod (netns) and applications can bind against the netkit device + queue



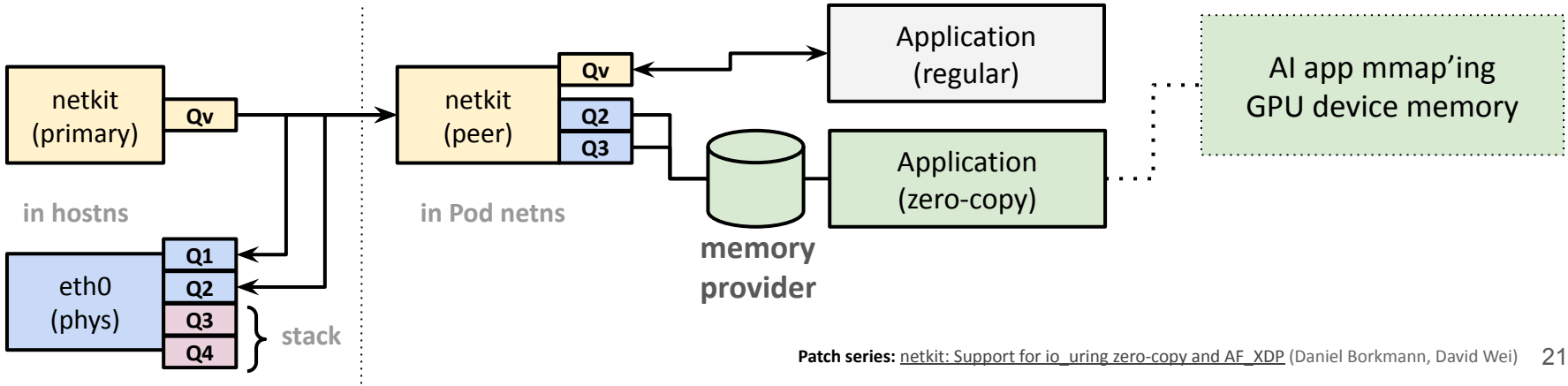
Patch series: netkit: Support for io_uring zero-copy and AF_XDP (Daniel Borkmann, David Wei)

Proposal: netkit & Hardware Queue Leasing for Pods



netkit extension for *native* zero-copy performance inside Pods:

- Create netkit peer with numrxqueues > 1 as *virtual queues*
- Lease physical queues in the physical NIC to virtual queues in netkit
- The netkit device is moved into the target Pod (netns) and applications can bind against the netkit device + queue
- This opens up *native* speed for AF_XDP zero-copy, io_uring & devmem zero-copy

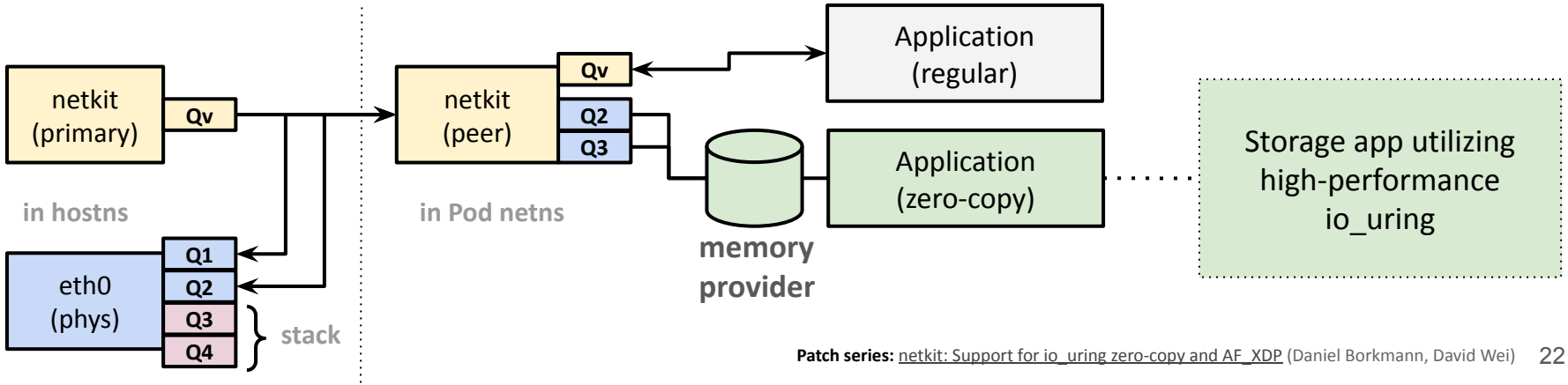


Patch series: netkit: Support for io_uring zero-copy and AF_XDP (Daniel Borkmann, David Wei)

Proposal: netkit & Hardware Queue Leasing for Pods

netkit extension for *native* zero-copy performance inside Pods:

- Create netkit peer with numrxqueues > 1 as *virtual queues*
- Lease physical queues in the physical NIC to virtual queues in netkit
- The netkit device is moved into the target Pod (netns) and applications can bind against the netkit device + queue
- This opens up *native* speed for AF_XDP zero-copy, io_uring & devmem zero-copy



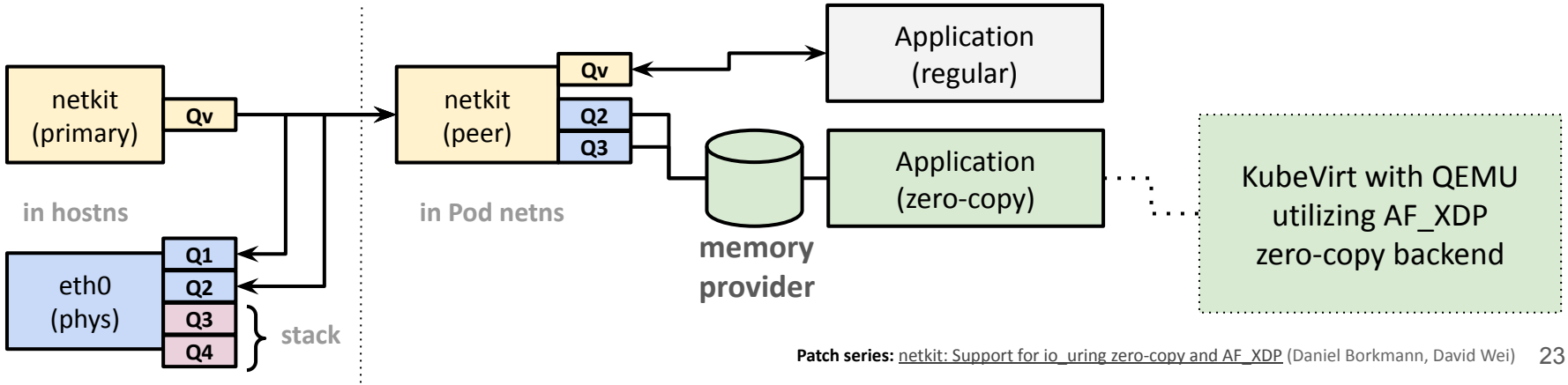
Patch series: netkit: Support for io_uring zero-copy and AF_XDP (Daniel Borkmann, David Wei)

Proposal: netkit & Hardware Queue Leasing for Pods



netkit extension for *native* zero-copy performance inside Pods:

- Create netkit peer with numrxqueues > 1 as *virtual queues*
- Lease physical queues in the physical NIC to virtual queues in netkit
- The netkit device is moved into the target Pod (netns) and applications can bind against the netkit device + queue
- This opens up *native* speed for AF_XDP zero-copy, io_uring & devmem zero-copy



Patch series: netkit: Support for io_uring zero-copy and AF_XDP (Daniel Borkmann, David Wei)

Same Queue Leasing, but Different Zero-Copy Use Cases

AF_XDP zero-copy

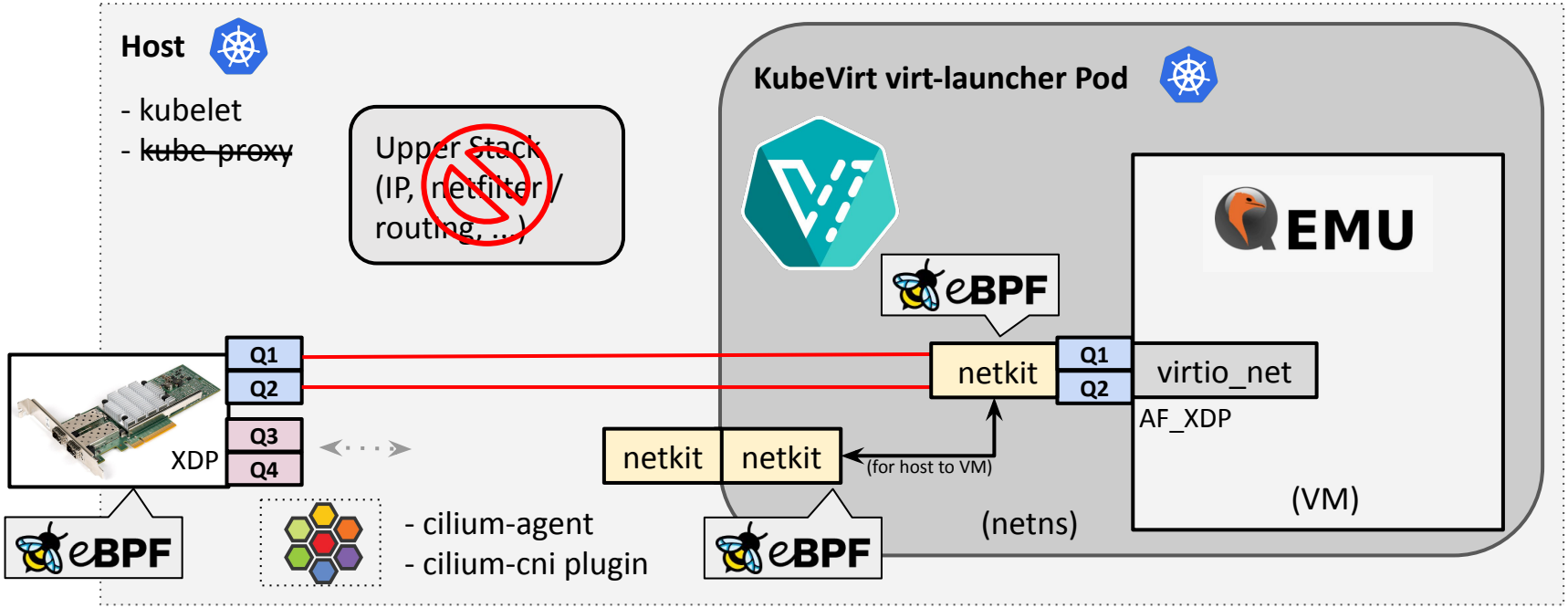
- Full control of everything, e.g. useful for DPDK-like applications in user space
- Build-your-own load balancer, gateway, firewall, *QEMU* has a backend
- Upper kernel stack is bypassed, but reuses driver, netdevice, XDP

io_uring with TCP zero-copy support

- Shift towards supporting header/data split for higher-end NICs (bnxt/mlx5/gve)
- Data (payload) pages mmap'ed with user space application or GPU device memory
- Upper kernel stack is *reused*, not bypassed - packet traverses up the normal stack

How can the netkit queue-binding & AF_XDP be applied for KubeVirt Pods?

Cilium's Future KubeVirt Pod Datapath:

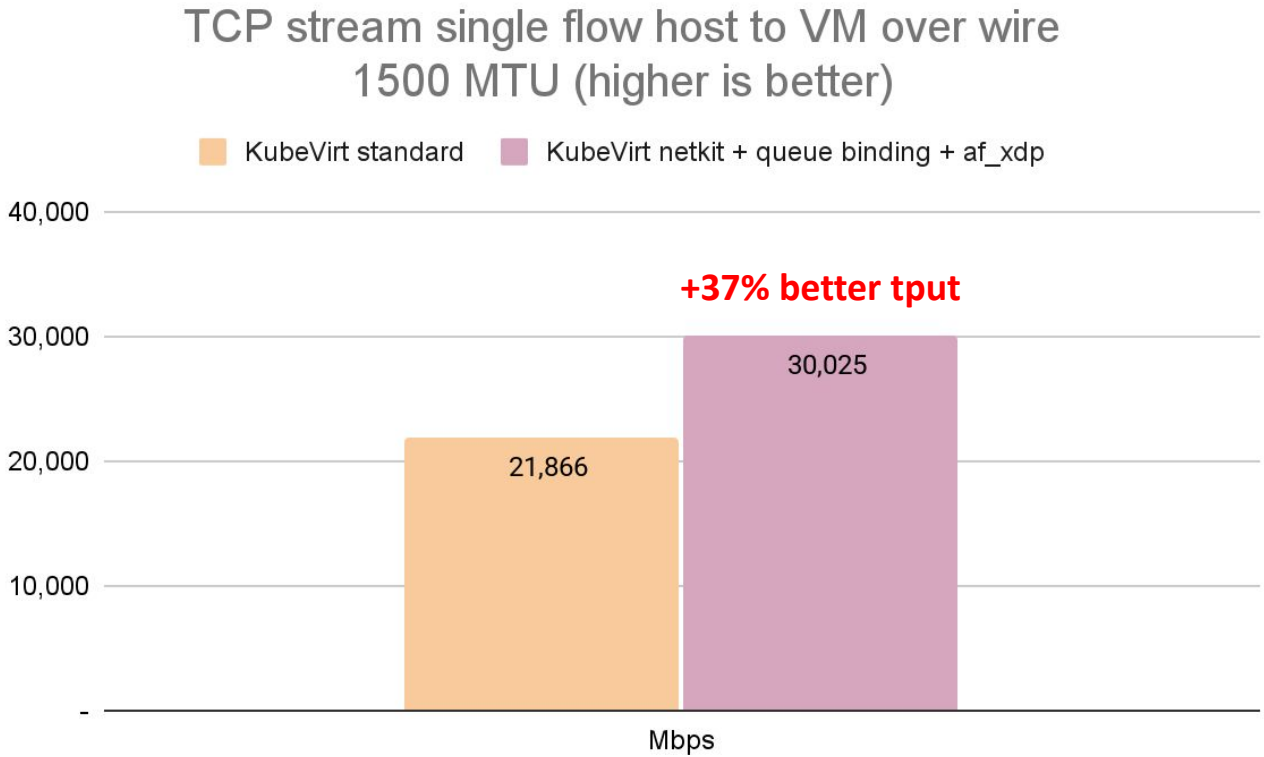


Preliminary benchmarks with netkit + AF_XDP for KubeVirt look promising! Note that QEMU's AF_XDP backend is still not full zero-copy (yet).

Cilium's Future K8s KubeVirt Pod Datapath:

Building Blocks:

- Cilium sets up new RSS context based on VM MAC
- Creates netkit device and binds netkit to phys queue
- Cilium KubeVirt plugin to launch QEMU with af-xdp bound to netkit



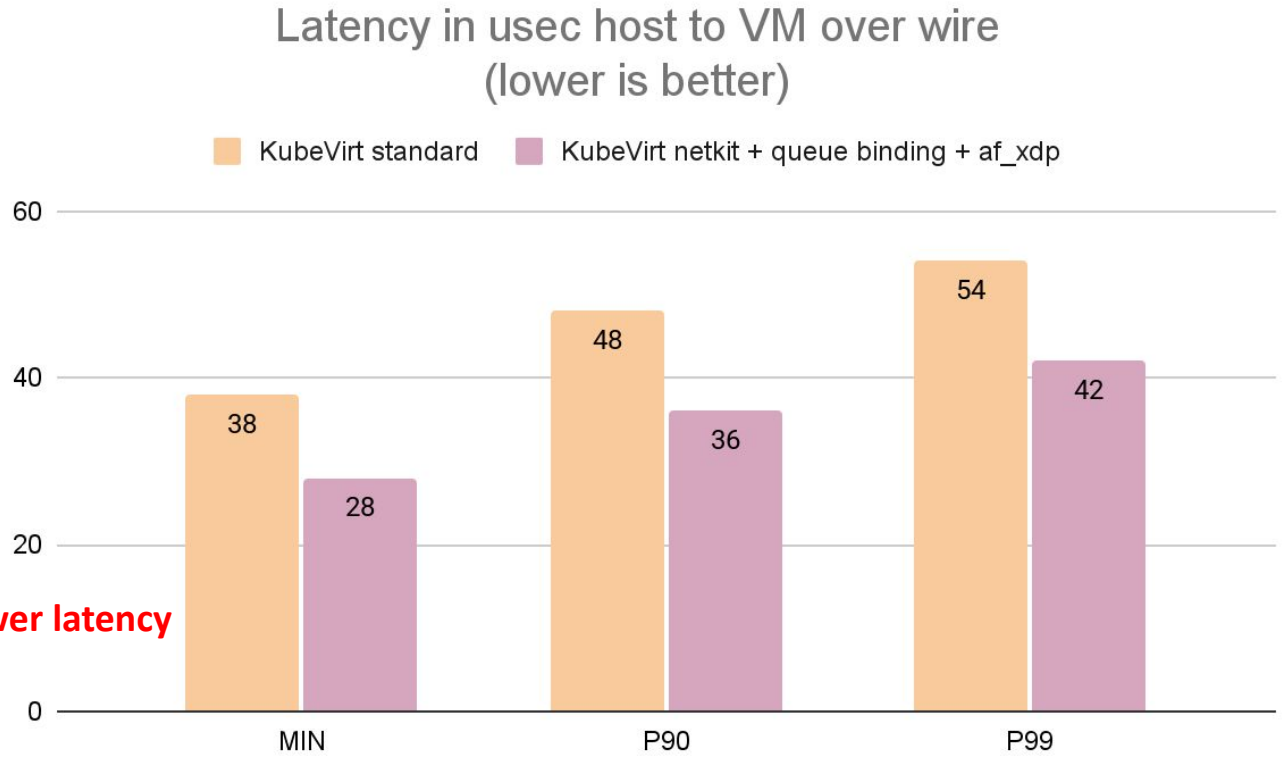
Back to back: AMD Ryzen 9 3950X @ 3.5 GHz, 128G RAM @ 3.2 GHz, PCIe 4.0, ConnectX-6 Dx, mlx5 driver, striding mode, LRO off, 1500 MTU
netperf -t TCP_RR -H <VM IP> -- -O MIN_LATENCY,P90_LATENCY,P99_LATENCY,THROUGHPUT

Cilium's Future K8s KubeVirt Pod Datapath:



Building Blocks:

- Cilium sets up new RSS context based on VM MAC
- Creates netkit device and binds netkit to phys queue
- Cilium KubeVirt plugin to launch QEMU with af-xdp bound to netkit



30-35% lower latency

Back to back: AMD Ryzen 9 3950X @ 3.5 GHz, 128G RAM @ 3.2 GHz, PCIe 4.0, ConnectX-6 Dx, mlx5 driver, striding mode, LRO off, 1500 MTU
netperf -t TCP_RR -H <VM IP> -- -O MIN_LATENCY,P90_LATENCY,P99_LATENCY,THROUGHPUT

Some implementation details (v5 series) ...

netdev.yaml extension

```
-
  name: queue-create
  doc: |
    Create a new queue for the given netdevice. Whether this operation
    is supported depends on the device and the driver.
  attribute-set: queue
  flags: [admin-perm]
  do:
    request:
      attributes:
        - ifindex
        - type
        - lease
    reply: &queue-create-op
      attributes:
        - id
```

netdev.yaml extension

```yaml
-
  name: queue-create
  doc: |
    Create a new queue for
    is supported depends o
  attribute-set: queue
  flags: [admin-perm]
  do:
    request:
      attributes:
        - ifindex
        - type
        - lease
    reply: &queue-create
      attributes:
        - id
```

```yaml
-
  name: lease
  attributes:
    -
      name: ifindex
      doc: The netdev ifindex to lease the queue from.
      type: u32
      checks:
        min: 1
    -
      name: queue
      doc: The netdev queue to lease from.
      type: nest
      nested-attributes: queue-id
    -
      name: netns-id
      doc: The network namespace id of the netdev.
      type: s32
```

```
-
  name: queue-get
  doc: Get queue information from the kernel.
       Only configured queues will be reported (as opposed to all available
       hardware queues).
  attribute-set: queue
  do:
    request:
      attributes:
        - ifindex
        - type
        - id
    reply: &queue-get-op
      attributes:
        - id
        - type
        - napi-id
        - ifindex
        - dmabuf
        - io-uring
        - xsk
        - lease
```



Device locking & refcounts

- To avoid deadlocks, we always ensure that virtual device is netdev_lock()'ed first, then the physical device to lease the queue from

```
struct netdev_rx_queue {
        ...
        struct netdev_rx_queue          *lease;
        netdevice_tracker               lease_tracker;
} ____cacheline_aligned_in_smp;
```

- Virtual device holds refcount on physical device upon leasing
- Queue stays leased until virtual device destruction
- Queue resizing blocked if leased queues affected
- netdev notifier to watch for physical device link removal to also remove all associated virtual devices (1:n)

New ndo for queue creation

- Virtual devices like netkit can implement the new queue management ndo

```
struct netdev_queue_mgmt_ops {
        ...
        int                     (*ndo_queue_create)(struct net_device *dev);
};
```

- netkit implementation allocates “real” queue, creates sysfs entries, and returns queue id which the caller later uses for queue leasing

Binding against memory providers

- User APIs unchanged
- Pass in netkit peer ifindex + virtual queue ids
- Transparently passed to physical queues

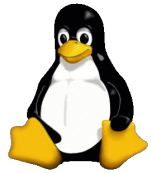
Demo: Cilium and KubeVirt with netkit’s queue leasing and AF_XDP QEMU backend ...

```
+ yq -C .spec.template < vm.yaml | head -20
{
  "metadata": {
    "labels": {
      "kubevirt.io/vm": "vm-net-binding-netkit"
    },
    "annotations": {
      "cilium.io/bind-queue": "enp2s0f1np1/15",
      "cilium.io/mac": "18:c0:4d:77:88:99"
    }
  },
  "spec": {
    "domain": {
      "devices": {
        "interfaces": [
          {
            "name": "netkit",
            "binding": {
              "name": "netkit"
            }
          }

.................
```

Next step is to tackle an XDP API rework since we need an AF_XDP TX hook for QEMU (policy enforcement, visibility, etc)

Potential XDP API changes



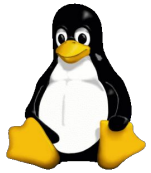
The original reason:

- “XDP TX”: for the zero-copy case the TX path is invisible to the host, but there might be need to observe/policy traffic

And, while we’re here:

- Use bpf_mprog to allow attaching multiple XDP programs per device in a similar manner to tcx
- Allow attaching XDP programs per queue / RSS context

XDP multi-prog



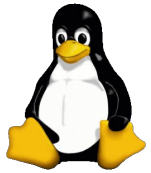
There were previously discussions on how implement multi-attach capabilities for XDP:

- XDP: the Distro View, 2019
- Multiple XDP programs on a single interface, 2020

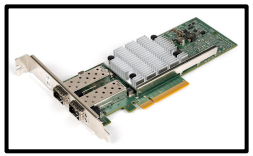
And the result was that Toke implemented support in userspace via freplace.

Since then Daniel added bpf_mprog interface, which '*can be reused by different attachment layers to enable multi-program attachment*'. So the idea is to reuse it for XDP to mimic TCX behaviour / user experience.

Current XDP [link-based] API



At the moment the BPF_XDP link attach type can be used to attach programs to device. In XDP case link->flags determine the attachment point – HW_MODE, DRV_MODE, SKB_MODE. Only two links can exist simultaneously (DRV_MODE & SKB_MODE can't be used at the same time):



DRV_MODE

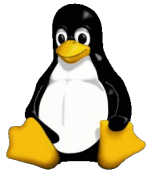
SKB_MODE

prog1 ← link1

HW_MODE

prog2 ← link2

New XDP bpf_mprog-based API



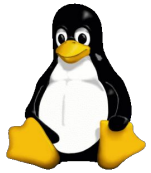
We propose to add two new attachment types by analogy with TCX:

- BPF_XDP_INGRESS
- BPF_XDP_EGRESS

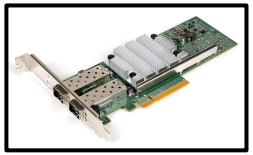
The command format in comparison with the old BPF_XDP has additional set of parameters:

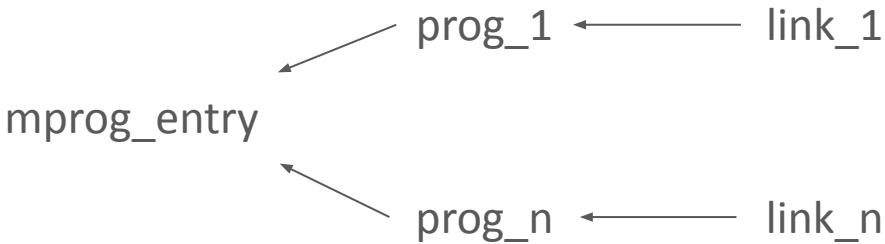
```
struct {
        union {
                __u32   relative_fd;
                __u32   relative_id;
        };
        __u64           expected_revision;
} xdp;
```

New XDP bpf_mprog-based API

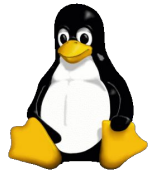


With this new API it is possible to attach multiple programs. The `relative_{fd,id}` together with `link->flags` can be used to choose a position of a new prog (see [patch] for the details on link flags to be used)

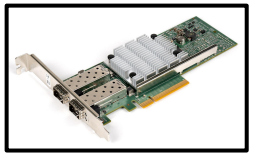


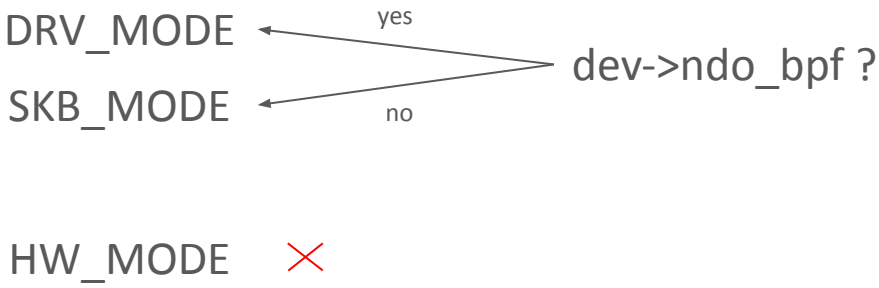


New XDP bpf_mprog-based API

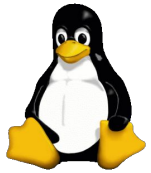


One question here: the tcx link attach link->flags conflicts with old xdp-style link->flags, so should we respect both (by adding more flags), or switch to selecting attachment point automatically (say, if device has ndo_bpf, then choose DRV_MODE, otherwise choose SKB_MODE; and totally ignore HW_MODE):



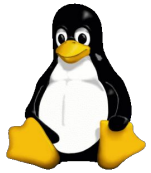


New XDP bpf_mprog-based API



The next question is: is this possible to support this new API with minimal drivers changes for BPF_XDP_INGRESS? For BPF_XDP_EGRESS the answer is, obviously, no. For BPF_XDP_INGRESS the answer is: “maybe”.

New XDP bpf_mprog-based API

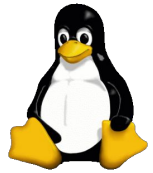


The XDP_SETUP_PROG command of ndo_bpf looks like this:

```
struct {
        u32 flags;
        struct bpf_prog *prog;
        struct netlink_ext_ack *extack;
};
```

The bpf_prog passed to drivers and, afaics, is dereferenced to do bpf_prog_{inc,put} and to check for prog->aux->xdp_has_frags.

bpf_mprog: Fast Path API

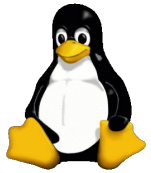


With bpf_mprog (in the fast path) the device sees a pointer to struct bpf_mprog_entry.
To run the program the code similar to following (from tcx_run) is supposed to be used:

```
bpf_mprog_foreach_prog(entry, fp, prog) {
        bpf_compute_data_pointers(skb);
        ret = bpf_prog_run(prog, skb);
        if (ret != TCX_NEXT)
                break;
}
```

This means that to support BPF_XDP_INGRESS new ndo_bpf is required and all drivers needs to be patched. Is there another way?

XDP / bpf_mprog: Let’s add more BPF



What instead of passing a pointer to mprog_entry and running a loop a new bpf program is created every time mprog structure is mangled. The pseudo-code for this wrapper program would look like this:

```
ctx = R1
call prog1
if r0 != NEXT exit
```

Save ctx, call the first program, if the return value is not “run the next”, exit.

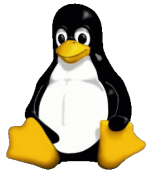
```
R1 = ctx
call prog2
if r0 != NEXT exit
...
```

Restore ctx, call the second program, if the return value is not “run the next”, exit.

```
R1 = ctx
call progN
exit
```

Do the same until the last program, exit with any value.

XDP / bpf_mprog: Let’s add more BPF



The real code for one chunk looks as follows. Here a new 16-bytes BPF_DIRECT_CALL instruction lets to pass a function address directly.

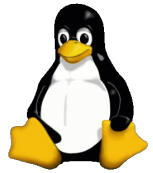
```
if (first)
        insn[i++] = BPF_MOV64_REG(BPF_REG_6, BPF_REG_1);
else
        insn[i++] = BPF_MOV64_REG(BPF_REG_1, BPF_REG_6);

insn[i++] = BPF_RAW_INSN(BPF_DIRECT_CALL, 0, 0, 0, (u32)func);
insn[i++] = BPF_RAW_INSN(0, 0, 0, 0, (u32)(func >> 32));

if (!last)
        insn[i++] = BPF_JMP_IMM(BPF_JEQ, BPF_REG_0, next_rc, 1);

insn[i++] = BPF_EXIT_INSN();
```

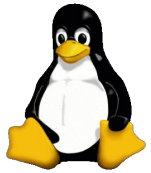
XDP / bpf_mprog: JIT Part



The jit patch for this new instruction is straightforward (a few non-important lines removed):

```
case BPF_DIRECT_CALL: {
        …
        func = (u8 *)(long)((u64)insn[1].imm << 32 | (u32)insn[0].imm);
        if (emit_call(&prog, func, ip))
                return -EINVAL;
        …
        break;
}
```

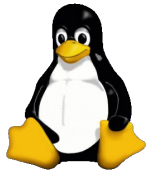
XDP / bpf_mprog: required drivers changes



After that program code is constructed, bpf JIT is executed and the resulting BPF program can be safely passed to drivers, as before. (And if bpf_mprog contains only one program, then nothing needs to be done at all.)

All the drivers will have to be patched, as by analogy with TCX a new return code XDP_NEXT code should be added. However, changes are trivial, as from a driver's perspective XDP_NEXT acts the same as XDP_PASS.

New XDP Attach API: XDP_EGRESS



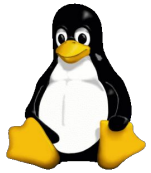
The API changes are as follows. First, add a new ndo_bpf command (and a TX XDP dispatcher)

```
/* XDP_SETUP_EGRESS_PROG */
struct {
        u32 flags;
        struct bpf_prog *prog;
        struct netlink_ext_ack *extack;
};
```

(This also transparently supports bpf_mprog.)

A new command is better than extending flags, because drivers don't check flags, and this is hard to find if driver supports new attachment type or not.

New XDP Attach API: XDP_EGRESS



Then extend xdp_md with

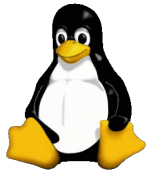
```
struct xdp_md {
        __u32 data;
        __u32 data_end;
        __u32 data_meta;

        __u32 ingress_ifindex; /* rxq->dev->ifindex */
        __u32 rx_queue_index;  /* rxq->queue_index  */

        __u32 egress_ifindex;  /* txq->dev->ifindex */
        __u32 tx_queue_index;  /* txq->queue_index  */
};
```

The new field will only be mapped for BPF_XDP_EGRESS attach type.

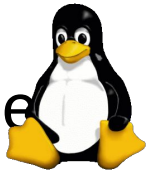
New XDP Attach API: XDP_EGRESS



However, the question with BPF_XDP_EGRESS is: what should be the exact attachment point? There could be several options (with our use case being the first one):

- Only those generated by AF_XDP (for example, for mlx5 this would correspond to MLX5E_XDP_XMIT_MODE_XSK)
- More specific attachment points (+ xdp_redirect, etc.)
- All packets

New XDP Attach API: Per queue / RSS Context Attachme



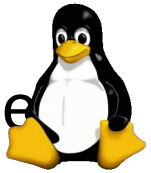
This could be a good optimization to execute XDP programs per-queue.

In our use-case, for example, we're interested in running XDP RX/TX only on those queues where such traffic may appear.

The remaining traffic should *not* pay the cost of entering the BPF program in the first place.

This change, however, adds a bunch of complexity to the API, e.g. atomic BPF program updates for a set of queues.

New XDP Attach API: Per queue / RSS Context Attachme

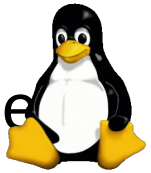


The API changes are to add a new command:

```
/* XDP_SETUP_INGRESS_PROG */
struct {
        u32 flags;
        struct bpf_prog *prog;
        struct netlink_ext_ack *extack;
        u32 queue_start;
        u32 queue_end;
};
```

Additional complexity: there are now two ways to attach XDP: “default” (all queues) and “per-rss” (queue range). These two cases shouldn’t be mutually exclusive.

New XDP Attach API: Per queue / RSS Context Attachme



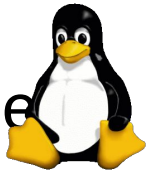
For example, a device has 8 queues

[∅,∅,∅,∅,∅,∅,∅,∅]	The device has no programs attached
[∅,∅,∅,∅,∅,∅,x,x]	A program x is linked to queues 6-7
[y,y,y,y,y,y,x,x]	A program y is attached to "all" queues
[y,y,y,y,y,z,x,x]	A program z is attached to queue 5
[∅,∅,∅,∅,∅,z,x,x]	The default program y is detached
[∅,∅,∅,∅,∅,z,∅,∅]	The program x is detached

Links can't intersect:

[∅,∅,∅,∅,∅,z,∅,∅]	An attempt is done to attach x at 4-7: returns -ESMTH

New XDP Attach API: Per queue / RSS Context Attachme



Let's consider some more cases:

`[∅,∅,∅,∅,∅,∅,∅,∅]` The device has no programs attached

`[∅,∅,∅,∅,∅,∅,x,x]` A program x is linked to queues 6-7

`[y,y,y,y,y,y,x,x]` A program y is attached to queues 0-5 (not "all" queues)

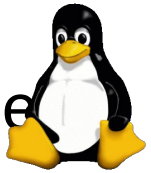
Now, try to attach a "default" program z:

`[y,y,y,y,y,y,x,x]` A program z is attached to "all" queues

What happens when x detached? It should be repopulated with default:

`[y,y,y,y,y,y,z,z]` A program x is detached

New XDP Attach API: Per queue / RSS Context Attachme

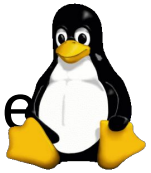


Another potential problem is XDP Dispatcher, which is a mechanism to avoid indirect calls (see the [patch series] for documentation and performance benchmarks).

One problem with the dispatcher is that it is not magic to translate indirect call to direct, and it has a limitation on the number of programs which can be speed up. This is 64 for the time being.

This is ok if there is 1:1 correspondence between device and program, but if a program can be attached to every queue, the number 64, theoretically, looks too low, as each device on the node can potentially have hundreds queues.

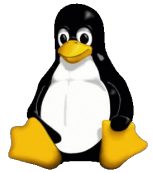
New XDP Attach API: Per queue / RSS Context Attachme



A few ideas how to deal with dispatcher

1. Increase dispatcher size (defined statically)
2. Introduce separate RX and TX dispatchers, as how probable could it be to run same programs on RX/TX
3. Introduce “dynamic” dispatcher, say, per-device. This, however, is not obvious yet if this is possible to do without an indirect call. And in this case, a fallback from a static dispatcher is preferrable

Legacy API vs. New API



In case of XDP we have “legacy API”, but also “legacy legacy API”. The previous two APIs are incompatible: either prog attachment or link attachment could be used. So, the “new” API will not definitely work with the “legacy legacy”.

For the compatibility between BPF_XDP & BPF_XDP_INGRESS links, looks like this is also easier to make them incompatible. To be 100% clear:

- If BPF_XDP link was used, do not allow BPF_XDP_INGRESS
- If BPF_XDP_INGRESS was used, do not allow BPF_XDP

(Of course, this doesn’t apply to BPF_XDP_EGRESS, it’s independent from the RX side.)

Thank you! Questions?

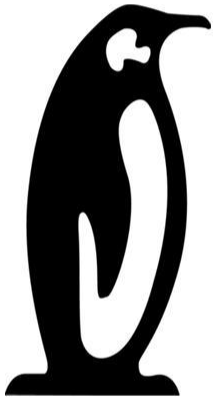
netkit upstream driver

netkit: Support for io_uring zero-copy and AF_XDP [v4]

XDP rework PoC branch

ISOVALENT
now part of cisco

Meta



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

